Exercise 2 - Hints and solutions

The **hints** come first. Try to solve each step with the help of the hints.

The **solutions** are at the bottom of this notebook. Look at them only after you have tried!

Hints

Part 1 - Comments and docstrings

1. What does the function calculate?

- Look at the `if` line: which letters are counted?
- Look at the `return` line: what does `n / len(s) * 100` give?

Does that match the comment `# count the G's`? Remember: a wrong comment is worse than no comment!

2. Rewrite the function

- Change `def f(s):` to `def at_content(sequence):` and replace `s` with `sequence` everywhere in the function. You can also give `n` and `x` better names, like `count` and `base`.
- A docstring is written in triple quotes `"""..."""`, as the **first line inside** the function:

```
def at_content(sequence):
    """
    One sentence: what the function does.

    Parameters:
        sequence: ...

    Returns:
        ...
    """
```

3. Check your docstring

Run `help(at_content)`.

Part 2 - Handling errors

1. Convert every value

```
for length in read_lengths:
    number = int(length)
```

The last line of the error message tells you the type of error, e.g. `ValueError: invalid literal for int() ...`. This is the name you will write after `except`.

2. Skip the broken values

Put only the line that can fail inside `try`, and handle the error in `except`:

```
try:
    number = int(length)
except ValueError:
    print(...)
    continue      # go to the next value in the loop
```

3. The average

- Before the loop, create an empty list `valid_lengths = []`
- After a successful conversion, add the number to the list with `.append()`
- After the loop: average = `sum(valid_lengths) / len(valid_lengths)`

Part 3 - User input

1. The type of the input

`input()` **always** returns a string, even if the user types a number. Check it with `type()`.

2. Ask until you get an integer

Use a `while` loop. Start with a variable that says "no valid number yet":

```
min_length = None

while min_length is None:
    answer = input("Give a minimum read length: ")
    # try to convert answer with int()
    # if it fails, print a message (min_length stays None, so the loop asks again)
```

Remember that `input()` must be **inside** the loop, otherwise the user is never asked again!

3. Filter the reads

Create an empty list, loop over `valid_lengths` from Part 2, and `append` the lengths that are `>= min_length`.

Solutions

Part 1 - Comments and docstrings

1. The function counts the `A` and `T` bases (not the `G`s!) and returns their **percentage** in the sequence. The comment is wrong.

In [ ]:
```
def f(s):
    # count the G's
    n = 0
    for x in s:
        if x == 'A' or x == 'T':
            n += 1
    return n / len(s) * 100

f('ATGCAT')
```

2.

In [ ]:
```
def at_content(sequence):
    """
    Calculate the AT content of a DNA sequence.

    Parameters:
        sequence: DNA sequence as a string, e.g. 'ATGCAT'

    Returns:
        The percentage of A and T bases in the sequence, as a float.
    """
    count = 0
    for base in sequence:
        if base == 'A' or base == 'T':
            count += 1
    return count / len(sequence) * 100

at_content('ATGCAT')
```

3.

In [ ]:
```
help(at_content)
```

Part 2 - Handling errors

1. The loop stops at `'N/A'` with:

```
ValueError: invalid literal for int() with base 10: 'N/A'
```

2. and 3.

In [ ]:
```
read_lengths = ['150', '148', '151', 'N/A', '150', 'error', '149']

valid_lengths = []

for length in read_lengths:
    try:
        number = int(length)
    except ValueError:
        print(f"Skipping value: {length}")
        continue
    valid_lengths.append(number)

average = sum(valid_lengths) / len(valid_lengths)
print(f"Average read length: {average}")
```

Part 3 - User input

1. The type is always `str`:

In [ ]:
```
answer = input("Give a minimum read length: ")
type(answer)
```

2. and 3.

In [ ]:
```
min_length = None

while min_length is None:
    answer = input("Give a minimum read length: ")
    try:
        min_length = int(answer)
    except ValueError:
        print(f"'{answer}' is not an integer. Try again!")

long_reads = []
for length in valid_lengths:
    if length >= min_length:
        long_reads.append(length)

print(f"Reads with length >= {min_length}: {long_reads}")
```